

Dockerization Documentation

Gujarat Police Hackathon — Camera Management & Streaming System

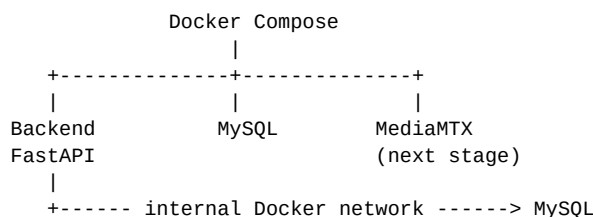
Purpose: Document the steps used to containerize the FastAPI backend and MySQL database, explain the Docker/Compose architecture, record the configuration created, and document the troubleshooting performed.

Document status: Backend + MySQL Docker deployment verified successfully. MediaMTX containerization is the next planned stage.

1. Project Goal

The original project depended on software installed directly on the Windows machine, including Python/virtual environments, MySQL, FFmpeg and MediaMTX. The objective of Dockerization is to move application dependencies into reproducible containers so another machine can clone the repository, install Docker, and start the project with Docker Compose.

The target architecture is:



2. Why Docker Was Chosen

Docker packages an application and its runtime dependencies into isolated containers. This avoids requiring every machine to reproduce the exact local Python, MySQL, FFmpeg and service configuration manually.

An important networking concept discovered during setup is that containers on the same Docker network can communicate directly by service name. Therefore the backend connects to MySQL using **database:3306**, not Windows localhost.

3. Existing Project Structure Relevant to Docker

```
GUJARAT_POLICE_HACKETHON/
├── backend/
│   ├── app/
│   ├── requirements.txt
│   ├── database/
│   │   ├── init.sql
│   │   └── deployment/
│   │       ├── docker-compose.yml
│   │       └── docker/
│   │           ├── backend.Dockerfile
│   │           ├── frontend.Dockerfile
│   │           └── mediamtx.Dockerfile
│   ├── frontend/
│   ├── streaming/
│   └── mediamtx/
│       └── mediamtx.yml
```

■■■ ...

Note: The repository also contains project documentation and application source code. The Docker deployment files are kept under **deployment/**.

4. Database Design Used for the Current Setup

The working database contains two application tables: **vendor** and **camera**. The camera table references vendor through VendorID.

Table	Important columns	Relationship
vendor	VendorID, VendorName, NUMBER_CAM, URL, enable	Parent table
camera	CameraID, CameraName, VendorID, CameraURL, enable	VendorID → vendor.VendorID

5. Backend Database Connection: Before vs. Docker

The original Python connection used localhost because the application previously ran directly on Windows:

```
mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="root",  
    database="gujarat_police"  
)
```

Inside Docker, **localhost** would mean the backend container itself. It would not mean the MySQL container. Therefore the Dockerized application should use the Compose service name:

```
host="database"  
user="root"  
password="root"  
database="gujarat_police"  
port=3306
```

The service name **database** is resolvable by Docker's internal network.

6. Docker Compose Configuration

Docker Compose describes multiple services and their relationships in one YAML file. The current deployment defines a backend service and a MySQL database service. The backend is exposed on host port 8000. MySQL does not need a host port because the backend communicates with it internally.

```
services:  
  backend:  
    build:  
      context: ..  
      dockerfile: deployment/docker/backend.Dockerfile  
    container_name: gujarat_police_backend  
    depends_on:  
      - database  
    environment:  
      DB_HOST: database  
      DB_NAME: gujarat_police  
      DB_PASSWORD: root  
      DB_USER: root  
    ports:  
      - "8000:8000"  
    restart: unless-stopped  
  
  database:  
    image: mysql:8.0  
    container_name: gujarat_police_database
```

```
environment:
  MYSQL_DATABASE: gujarat_police
  MYSQL_ROOT_PASSWORD: root
volumes:
  - mysql_data:/var/lib/mysql
  - ../database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
restart: unless-stopped
```

```
volumes:
  mysql_data:
```

Important: The exact final YAML in the working repository should remain the source of truth. The snippet above documents the architecture and intended configuration.

7. Understanding the Dockerfile

A Dockerfile is a recipe for creating an image. The image contains the runtime environment and dependencies needed by the application. A container is a running instance of that image.

For the FastAPI backend, the Dockerfile needs to establish a Python environment, install requirements, copy the project files, and start Uvicorn.

```
FROM python:3.12-slim

WORKDIR /app

COPY backend/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY backend ./backend

EXPOSE 8000

CMD ["uvicorn", "backend.app.main:app",
    "--host", "0.0.0.0", "--port", "8000"]
```

The key point is that the container runs Uvicorn on **0.0.0.0:8000**. Compose publishes that container port to the host so the browser can reach the API through **localhost:8000**.

8. Why the Compose Build Context Matters

A previous build error occurred because the Compose file resolved the Docker build context incorrectly. Docker reported that the referenced deployment directory could not be found.

The concept is:

```
build:
  context: ..
  dockerfile: deployment/docker/backend.Dockerfile
```

Here **..** means 'the parent directory of the directory containing docker-compose.yml'. If docker-compose.yml is in **deployment/**, then its project root is one level above it. The Dockerfile path is then interpreted relative to the build context.

This distinction is important: **build context** determines which files Docker is allowed to send to the builder; **dockerfile** tells Docker which Dockerfile to use.

9. Dependency Discovery and requirements.txt

The backend imports FastAPI, Pydantic, MySQL Connector, httpx, pandas and standard-library modules. During recovery from the hard-drive failure, the virtual environment was rebuilt and the database connector was initially missing.

```
The missing dependency produced:
```

```
ModuleNotFoundError: No module named 'mysql'
```

```
Installing the dependency fixed the test. The environment then reported:
```

```
mysql-connector-python==26.7.0
```

The dependency was added to `backend/requirements.txt` so Docker can install it reproducibly instead of depending on the developer's local virtual environment.

10. Database Initialization

Because the database structure needs to exist inside a fresh MySQL container, an SQL initialization file is used. This is preferable to manually running Python setup code for basic schema creation because SQL directly describes the database schema and can be automatically executed by the MySQL image on first initialization.

```
The initialization mechanism is mounted into:
```

```
/docker-entrypoint-initdb.d/init.sql
```

The MySQL image executes initialization scripts when its data directory is being initialized. Because a named volume persists the database, the initialization scripts are not automatically re-run every time the container restarts.

11. Docker Networking and Ports

The project encountered a port conflict when Docker attempted to publish MySQL on Windows port 3306. The error indicated that port 3306 was already in use by an existing Windows MySQL installation.

```
HOST PORT : CONTAINER PORT
```

```
"3307:3306"
```

```
Windows :3307 ---> Docker MySQL :3306
```

However, publishing MySQL to the Windows host is unnecessary when only the backend needs it. The backend and database can communicate directly inside the Docker network using **database:3306**. Therefore the final architecture does not need a MySQL host-port mapping unless an external tool such as MySQL Workbench needs direct access.

By contrast, port 8000 is intentionally published because the browser running on the host needs to reach FastAPI.

12. Commands Used During Recovery and Docker Setup

```
# Verify Docker
docker --version
docker compose version

# Validate Compose YAML
docker compose -f .\deployment\docker-compose.yml config

# Build and start
docker compose -f .\deployment\docker-compose.yml up --build

# Start without rebuilding
docker compose -f .\deployment\docker-compose.yml up

# Stop/remove Compose containers
docker compose -f .\deployment\docker-compose.yml down
```

```
# Show running containers
docker ps

# Test the FastAPI root endpoint
curl http://localhost:8000/

# Test FastAPI Swagger UI
curl http://localhost:8000/docs
```

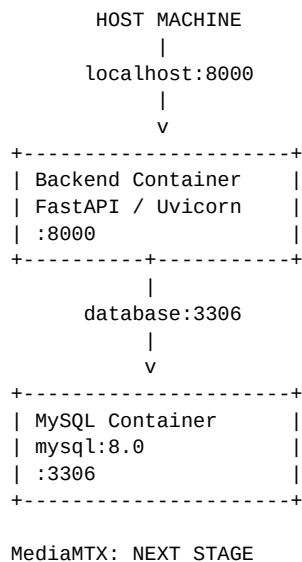
13. Verification Performed

The deployment was successfully verified with **docker ps**. The resulting containers were:

Container	Image	Status	Host exposure
gujarat_police_backend	deployment-backend	Up	0.0.0.0:8000 → 8000
gujarat_police_database	mysql:8.0	Up	Internal 3306/tcp

The FastAPI endpoint returned HTTP 200 OK and the response was served by Uvicorn. This confirms that the backend container was reachable from Windows.

14. Current Architecture



15. MediaMTX and FFmpeg — Planned Dockerization

The original streaming setup used MediaMTX and FFmpeg directly on Windows, including executable files and a Task Scheduler arrangement. The Dockerization plan is to move MediaMTX into its own container and manage streaming services through Compose.

The target design is:

```
Docker Compose
■■■ backend
■■■ database
■■■ mediamtx
    ■■■ RTSP
    ■■■ HLS
    ■■■ WebRTC
```

FFmpeg can then be treated as a container dependency or included in the streaming image depending on the final stream architecture. The exact approach should be chosen after validating the current MediaMTX configuration and how the FastAPI service launches/manages streams.

16. Portability Checklist

For another computer to run the project, the repository should contain the source code, requirements, Dockerfiles, Compose configuration, database initialization SQL, and MediaMTX configuration.

Machine-specific folders such as local virtual environments, downloaded Windows FFmpeg/MediaMTX directories, credentials and generated runtime data should not be required.

Recommended repository items:

- ✓ backend/requirements.txt
- ✓ deployment/docker-compose.yml
- ✓ deployment/docker/backend.Dockerfile
- ✓ database/init.sql
- ✓ streaming/mediamtx/mediamtx.yml
- ✓ application source code

Avoid committing:

- ✗ .venv/
- ✗ local executable folders
- ✗ secrets/passwords
- ✗ generated runtime databases
- ✗ machine-specific Task Scheduler configuration

17. Recovery Lesson After the Hard-Drive Failure

The project recovery demonstrated why source code alone is not sufficient for reproducibility. GitHub preserved the tracked source files, but locally installed tools and configurations were lost. Dockerization reduces this dependency by making the runtime environment part of the project's deployment definition.

The long-term goal is therefore not merely 'put everything in Docker', but to make the repository itself describe how the system is built, configured, connected, initialized and started.

18. Recommended Next Steps

1. Keep the verified backend + MySQL Compose setup stable.
2. Confirm the database initialization SQL contains the vendor and camera schema required by the application.
3. Add the MediaMTX service to Compose.
4. Test RTSP streaming from the MediaMTX container.
5. Decide where FFmpeg belongs in the final architecture and containerize it accordingly.
6. Test the entire repository on a clean machine using only Docker and the repository.
7. Update the README with one-command startup and troubleshooting instructions.

End of documentation — Backend and MySQL Docker stage